Name of the module: Radix 2 DIF FFT

Location in GIT repository: vspa_common\vspa-lib\diffft

Type of code:

- VCPU kernel functions (24 functions)

VCPU Function API:

```
extern void fftDIF<N>_hfx_sfl (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
        vspa_complex_float32*     pOut,             // Output data address.
        vspa_complex_fixed16 const*   pBase,        // Circular buffer base address.
        unsigned int  const  cbuffSize              // Size of circular buffer in half-words
        );

extern void ifftDIF<N>_hfx_sfl (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
        vspa_complex_float32*     pOut,             // Output data address.
        vspa_complex_fixed16 const*   pBase,        // Circular buffer base address.
        unsigned int  const cbuffSize               // Size of circular buffer in half-words
        );
extern void fftDIF<N>_hfx_hfl (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
        vspa_complex_float16*     pOut,             // Output data address.
        vspa_complex_fixed16 const*   pBase,        // Circular buffer base address.
        unsigned int  const  cbuffSize              // Size of circular buffer in half-words
        );

extern void ifftDIF<N>_hfx_hfl (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
        vspa_complex_float16*     pOut,             // Output data address.
        vspa_complex_fixed16 const*   pBase,        // Circular buffer base address.
        unsigned int  const cbuffSize               // Size of circular buffer in half-words
        );
extern void fftDIF<N>_hfx_hfx (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
        vspa_complex_fixed16*     pOut,             // Output data address.
        vspa_complex_fixed16 const*   pBase,        // Circular buffer base address.
        unsigned int  const  cbuffSize              // Size of circular buffer in half-words
        );

extern void ifftDIF<N>_hfx_hfx (
        vspa_complex_fixed16 const*   pIn,          // Input data address.
```

```
vspa_complex_fixed16*     pOut,          // Output data address.
vspa_complex_fixed16 const*   pBase,     // Circular buffer base address.
unsigned int  const cbuffSize            // Size of circular buffer in half-words
);
```

N = 64, 128, 256, 512

Implementation plan:

This module provides VCPU functions to perform decimation in frequency (DIF) FFT and IFFT of an input sequence of length as specified by the corresponding N.

The 4 functions are described below.

1. fftDIF<N>_hfx_sfl
   Accept inputs as half precision fixed point complex numbers and writes outputs as single precision floating point complex numbers.
   Mathematically, this function is equivalent to the following MATLAB command:
   y = fft(x, N);
   This is mathematically equivalent to, $y(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}$

2. ifftDIF<N>_hfx_sfl
   Accept inputs as half precision fixed point complex numbers and writes outputs as single precision floating point complex numbers.
   This function is equivalent to the following MATLAB command:
   y = N*ifft(x, N);
   This is mathematically equivalent to, $y(n) = \sum_{k=0}^{N-1} x(k)e^{\frac{j2\pi kn}{N}}$

3. fftDIF<N>_hfx_hfl
   Accept inputs as half precision fixed point complex numbers and writes outputs as half precision floating point complex numbers.
   Mathematically, this function is equivalent to the following MATLAB command:
   y = fft(x, N);
   This is mathematically equivalent to, $y(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}$

4. ifftDIF<N>_hfx_hfl
   Accept inputs as half precision fixed point complex numbers and writes outputs as half precision floating point complex numbers.
   This function is equivalent to the following MATLAB command:
   y = N*ifft(x, N);
   This is mathematically equivalent to, $y(n) = \sum_{k=0}^{N-1} x(k)e^{j2\pi kn/N}$

5. fftDIF<N>_hfx_hfx

Accept inputs as half precision fixed point complex numbers and writes outputs as half precision fixed point complex numbers.
Mathematically, this function is equivalent to the following MATLAB command:
y = (1/N)*fft(x, N);
This is mathematically equivalent to, $y(k) = \frac{1}{N}\sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}$

6. ifftDIF<N>_hfx_hfx
   Accept inputs as half precision fixed point complex numbers and writes outputs as half precision fixed point complex numbers.
   This function is equivalent to the following MATLAB command:
   y = ifft(x, N);
   This is mathematically equivalent to, $y(n) = \frac{1}{N}\sum_{k=0}^{N-1} x(k)e^{j2\pi kn/N}$

The input buffer can be circular. There is no alignment restriction on the starting address of the input sequence. The output buffer has to be linear and the address must be aligned to the beginning of a DMEM line. The input sequence must be placed in memory in linear order. The output is in bit-reversed order.

DMEM requirements:

Function name	Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
fftDIF<N>_hfx_sfl	Input buffer	N	1	Caller
	Output buffer	2*N	32	Caller
ifftDIF<N>_hfx_sfl	Input buffer	N	1	Caller
	Output buffer	2*N	32	Caller
fftDIF<N>_hfx_hfl	Input buffer	N	1	Caller
	Output buffer	N	32	Caller
ifftDIF<N>_hfx_hfl	Input buffer	N	1	Caller
	Output buffer	N	32	Caller
fftDIF<N>_hfx_hfx	Input buffer	N	1	Caller
	Output buffer	N	32	Caller
ifftDIF<N>_hfx_hfx	Input buffer	N	1	Caller
	Output buffer	N	32	Caller

N is the FFT/IFFT size.

Target efficiency:
TBD

Test plan:

- **Feature that are going to be tested:**
  - Random Gaussian inputs